

Variables, Data Types and Input/Output in C++

[DEF] Definition to Remember

Variables in C++ are strongly-typed memory locations representing primitive and user-defined data. C++ standard I/O utilizes type-safe streams `std::cin` and `std::cout` with insertion/extraction operators alongside `std::getline`.

1. Concept Overview & Fundamentals

Variables, Data Types and Input/Output in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

* Ze

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> <code>push_back</code> , contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

* Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

Problem 1: AreaOfCircle

Problem Description & Objective:

C++ Code Implementation:

Implem

```

/*
 * Que
 * Cre
 */
/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 *
 * ==
 */
#include

```

```
using
//
int ma
//
do
do
//
co
//
ci
//
co
//
an
//
co
//
re
}
```

Problem 2: AreaOfSquare

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
using
```

```
//  
int ma  
//  
dc  
dc  
  
//  
co  
  
//  
ci  
  
//  
an  
  
//  
co  
  
//  
re  
}
```

Problem 3: Circumference

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
circum  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
using  
  
//  
int ma  
//
```

```
co
//
do
do

//
co

//
ci

//
ci

//
co

//
re
}
```

Problem 4: Declare2number

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 * ===
 */

#include

using

//
int ma
//
in
//
```

```
in
//
co

//
co

//
re
}
```

Problem 5: NameAge

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Def
 displa
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include
#include

using

//
int ma
//
st
st

//
in

//
co
//
ci
```

```
//  
co  
//  
ci  
  
//  
co  
//  
ci  
  
//  
co  
  
//  
re  
}
```

Problem 6: PatternPrint1

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
using  
  
//  
int ma  
//  
fo
```

```
}  
//  
re  
}
```

Problem 7: PatternPrint2

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
using  
  
//  
int ma  
//  
fo  
  
}  
//  
re  
}
```

Problem 8: SizeOf

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 and pr
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include

using

//
int ma
//
in
fl
dc
ch
bc

//
cc

//
cc

//
cc

//
re
}
```

Problem 9: Swap

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 *
 *
 * ==
 */

#include

using

//
int ma
//
in

//
co
//
ci

//
co
//
ci

//
co

//
//
te

//
a

//
```

```
b
//
cc
//
re
}
```

Problem 10: Welcome

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
#include

using

//
int ma
//
st

//
cc

//
ge

//
cc

//
re
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. Type-safe stream I/O using `std::cin >>` and `std::cout <<` eliminates format specifier mismatch errors.
2. `std::endl` outputs a newline character and forces a stream buffer flush.
3. `std::getline(std::cin, str)` captures full text lines including whitespaces.
4. Use `const` and `constexpr` for compile-time constant immutability and optimization.
5. Memory sizes: `int` (4 bytes), `float` (4 bytes), `double` (8 bytes), `char` (1 byte), `bool` (1 byte).

[RECALL] Quick Recall

`std::cin >> input` -> Type Checking -> Stream Extraction -> Logic Execution -> `std::cout << result`